

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ им.
Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по домашнему заданию
“Бот AnkiGram”

Выполнил:
Студент группы ИУ5-34Б
Мальков Н.А.
Преподаватель:
Нардид А.Н.

Москва 2025

Задание

1. Выберите язык программирования (который Вы ранее не изучали) и (1) напишите по нему реферат с примерами кода или (2) реализуйте на нем небольшой проект (с детальным текстовым описанием).
2. Реферат (проект) может быть посвящен отдельному аспекту (аспектам) языка или содержать решение какой-либо задачи на этом языке.
3. Необходимо установить на свой компьютер компилятор (интерпретатор, транспилятор) этого языка и произвольную среду разработки.
4. В случае написания реферата необходимо разработать и откомпилировать примеры кода (или модифицировать стандартные примеры).
5. В случае создания проекта необходимо детально комментировать код.
6. При написании реферата (создании проекта) необходимо изучить и корректно использовать особенности парадигмы языка и основных конструкций данного языка.
7. Приветствуется написание черновика статьи по результатам выполнения ДЗ. Черновик статьи может быть подготовлен группой студентов, которые исследовали один и тот же аспект в нескольких языках или решили одинаковую задачу на нескольких языках.

Описание, тексты программ

Мною были изучены API Telegram, ANKI и ANKI_Connect, были использованы навыки работы с html.

bot.py

```
# bot.py
import os
import re
import base64
import traceback
from html import unescape
from dotenv import load_dotenv
from bs4 import BeautifulSoup
import telebot
from telebot.types import InlineKeyboardMarkup, InlineKeyboardButton
from anki_connect import invoke

def parse_media_from_html(html: str) -> dict:
    if not html:
        return {'images': [], 'audios': [], 'videos': [], 'text': ''}

    html = unescape(html)
    soup = BeautifulSoup(html, 'html.parser')

    images = [img['src'] for img in soup.find_all('img') if img.get('src')]
    text_content = soup.get_text(separator=' ')
    media_matches = re.findall(r'\[(sound|video):([^\]]+)\]', text_content)
    audios = [m[1] for m in media_matches if m[0] == 'sound']
    videos = [m[1] for m in media_matches if m[0] == 'video']

    cleaned_html = re.sub(r'<img[^>]*>', '', html)
    cleaned_html = re.sub(r'\[(sound|video):[^\]]+\]', '', cleaned_html)
    cleaned_text = BeautifulSoup(cleaned_html, 'html.parser').get_text(separator=' ')
    cleaned_text = cleaned_text.strip()

    return {
        'images': images,
        'audios': audios,
        'videos': videos,
        'text': cleaned_text
    }

def safe_basename(filename: str) -> str:
    return os.path.basename(filename)

def retrieve_and_save(filename: str, save_name: str) -> bool:
```

```

    try:
        data_b64 = invoke('retrieveMediaFile', filename=filename)
        if not data_b64:
            return False
        with open(save_name, 'wb') as f:
            f.write(base64.b64decode(data_b64))
        return True
    except Exception:
        return False

user_sessions = {}

def main():
    load_dotenv()
    bot = telebot.TeleBot(os.getenv("BOT_TOKEN"))

    @bot.message_handler(commands=['start', 'help'])
    def send_welcome(message):
        bot.reply_to(message, "Привет! Используй /get_decks, чтобы выбрать колоду для тренировки.")

    @bot.message_handler(commands=['get_decks'])
    def get_decks(message):
        user_id = message.from_user.id
        try:
            deck_names = invoke("deckNames")
            if not deck_names:
                bot.send_message(user_id, "Колоды не найдены.")
                return

            keyboard = InlineKeyboardMarkup()
            for deck in deck_names:
                keyboard.add(InlineKeyboardButton(text=deck,
callback_data=f"train_{deck}"))

            bot.send_message(user_id, "Мои колоды:", reply_markup=keyboard)
        except Exception as e:
            bot.send_message(user_id, "Ошибка при получении списка колод.")
            print("get_decks error:", e)
            traceback.print_exc()

    @bot.callback_query_handler(func=lambda call: call.data.startswith("train_"))
    def start_training(call):
        user_id = call.from_user.id
        deck_name = call.data[len("train_"):]
        try:
            card_ids = invoke("findCards", query=f"deck:{deck_name}")
            if not card_ids:
                bot.send_message(user_id, "В выбранной колоде нет карточек.")
                return

```

```

cards_info = invoke("cardsInfo", cards=card_ids)
user_sessions[user_id] = {
    'deck': deck_name,
    'cards': cards_info,
    'current_index': 0,
    'stats': {'Again': 0, 'Hard': 0, 'Good': 0, 'Easy': 0}
}
send_next_card(bot, user_id)
except Exception as e:
    print("start_training error:", e)
    traceback.print_exc()

def send_next_card(bot, user_id):
    session = user_sessions.get(user_id)
    if not session:
        bot.send_message(user_id, "Сессия не найдена. Используйте /get_decks.")
        return

    if session['current_index'] >= len(session['cards']):
        show_summary(bot, user_id)
        return

    card = session['cards'][session['current_index']]
    front_html = card['fields'].get('Front', {}).get('value', '')
    back_html = card['fields'].get('Back', {}).get('value', '')
    parsed_front = parse_media_from_html(front_html)
    parsed_back = parse_media_from_html(back_html)

    session['back_text'] = parsed_back['text']

    if parsed_front['text']:
        bot.send_message(user_id, f"

```

```

        with open(tmp_path, 'rb') as video:
            bot.send_video(user_id, video)
        os.remove(tmp_path)

    keyboard = InlineKeyboardMarkup()
    keyboard.add(InlineKeyboardButton("Показать ответ 🙈",
callback_data="show_answer"))
    bot.send_message(user_id, "Нажми, чтобы увидеть ответ:", reply_markup=keyboard)

@bot.callback_query_handler(func=lambda call: call.data == "show_answer")
def show_answer(call):
    user_id = call.from_user.id
    session = user_sessions.get(user_id)
    if not session:
        bot.send_message(user_id, "Сессия не найдена. Используй /get_decks.")
        return

    back_text = session.get('back_text', '(пусто)')
    spoiler = f"|||{escape_markdown(back_text)}|||"
    bot.send_message(user_id, spoiler, parse_mode="MarkdownV2")

# Кнопки оценок
keyboard = InlineKeyboardMarkup()
keyboard.row(
    InlineKeyboardButton("🗨 Again", callback_data="grade_Again"),
    InlineKeyboardButton("😬 Hard", callback_data="grade_Hard"),
    InlineKeyboardButton("😊 Good", callback_data="grade_Good"),
    InlineKeyboardButton("😌 Easy", callback_data="grade_Easy"),
)
bot.send_message(user_id, "Выбери, насколько легко было:", reply_markup=keyboard)

@bot.callback_query_handler(func=lambda call: call.data.startswith("grade_"))
def handle_grade(call):
    user_id = call.from_user.id
    grade = call.data[len("grade_"):]
    session = user_sessions.get(user_id)
    if not session:
        bot.send_message(user_id, "Сессия не найдена. Используй /get_decks.")
        return

    session['stats'][grade] += 1
    session['current_index'] += 1
    send_next_card(bot, user_id)

def show_summary(bot, user_id):
    session = user_sessions.get(user_id)
    if not session:
        return

    stats = session['stats']

```

```

total = sum(stats.values())
text = (
    f"📊 <b>Результаты тренировки</b>\n\n"
    f"🔄 Again: {stats['Again']}\n"
    f"😓 Hard: {stats['Hard']}\n"
    f"😊 Good: {stats['Good']}\n"
    f"😎 Easy: {stats['Easy']}\n"
    f"\nВсего карточек: {total}"
)
bot.send_message(user_id, text, parse_mode='HTML')
del user_sessions[user_id]

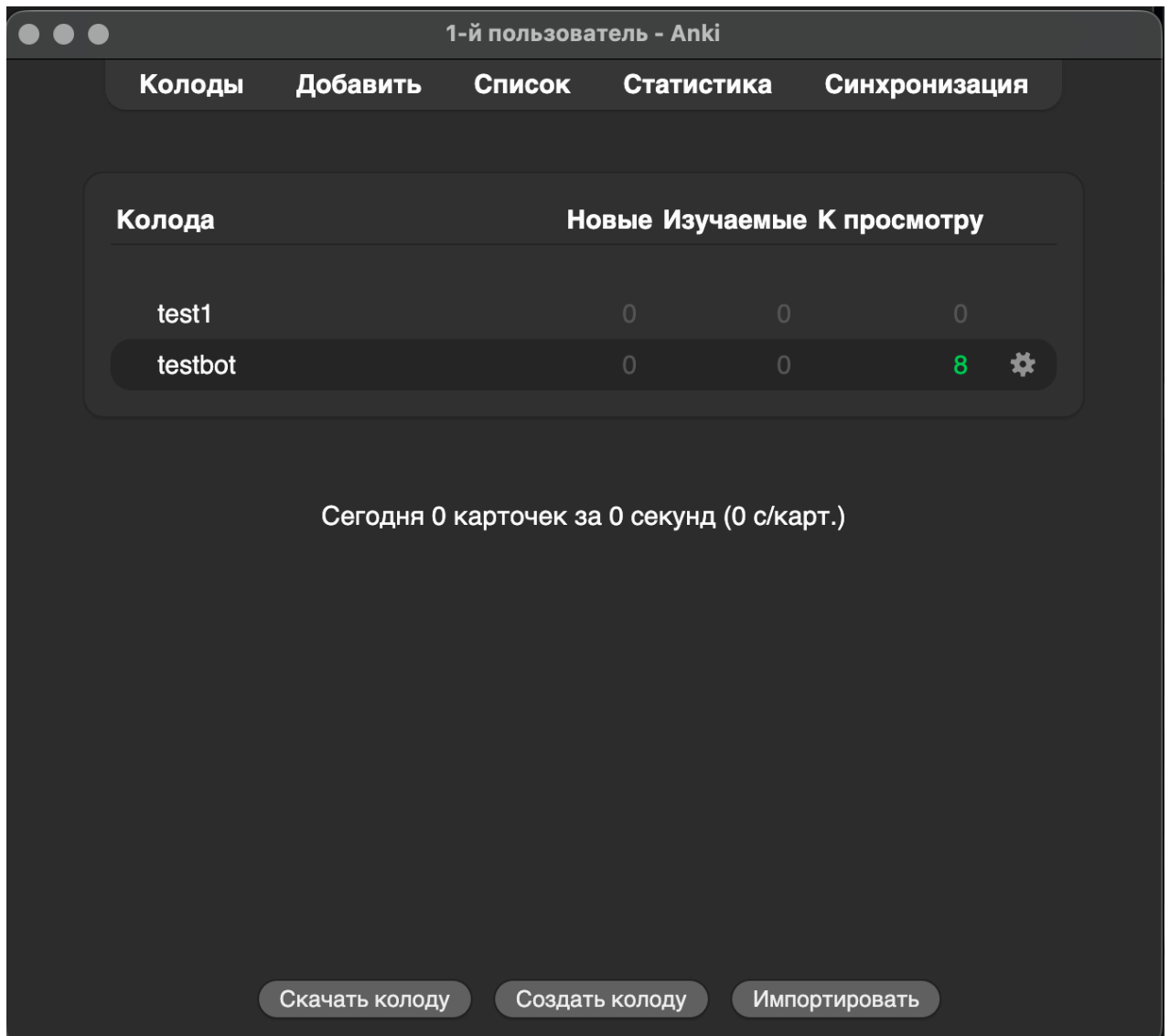
def escape_markdown(text: str) -> str:
    """Экранирует специальные символы для MarkdownV2."""
    escape_chars = r'_*[]()~`>#+-=|{ } . ! '
    return ''.join(f'\\{c}' if c in escape_chars else c for c in text)

bot.infinity_polling()

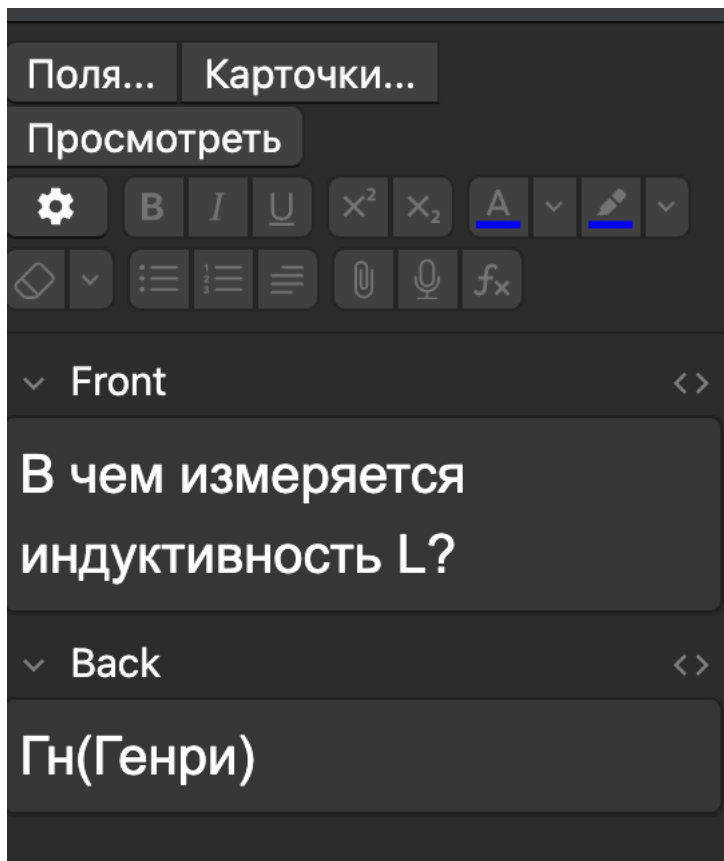
if __name__ == "__main__":
    main()

```

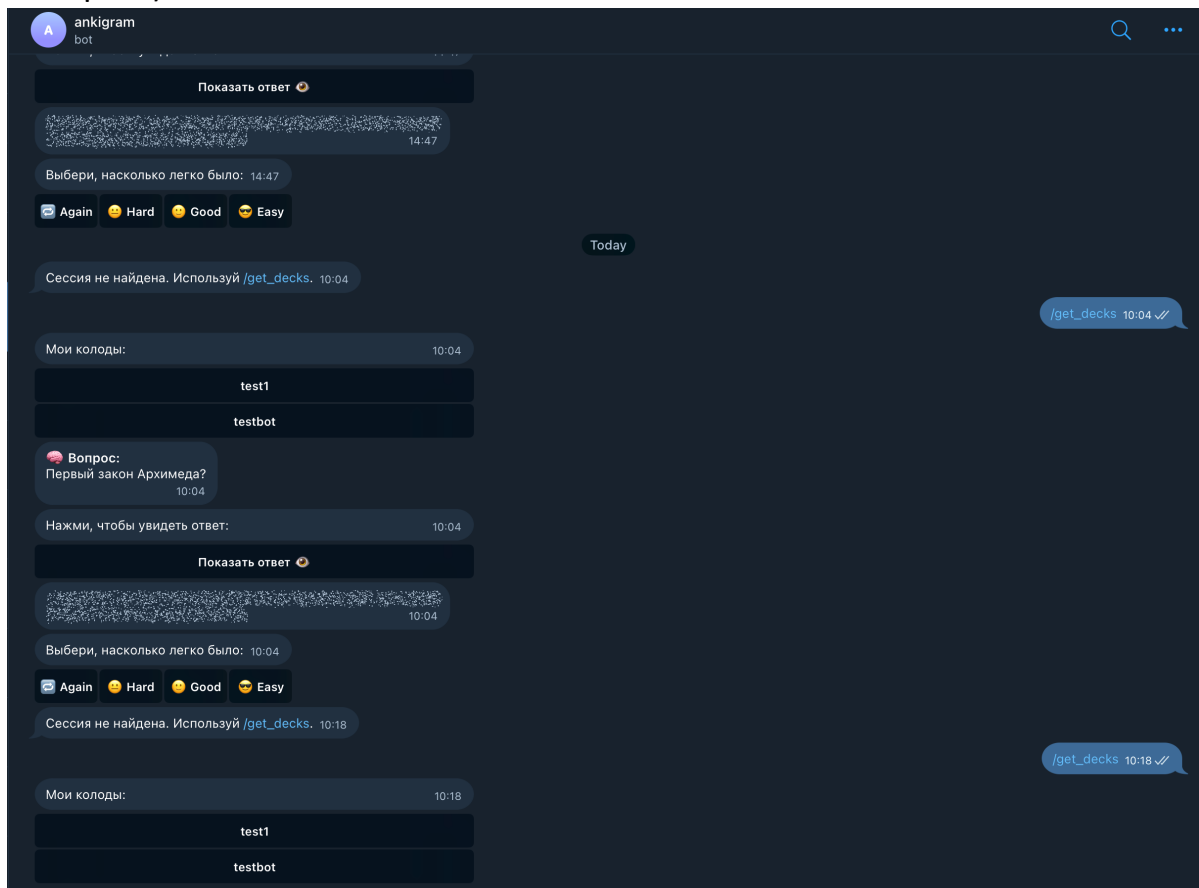
Работа бота



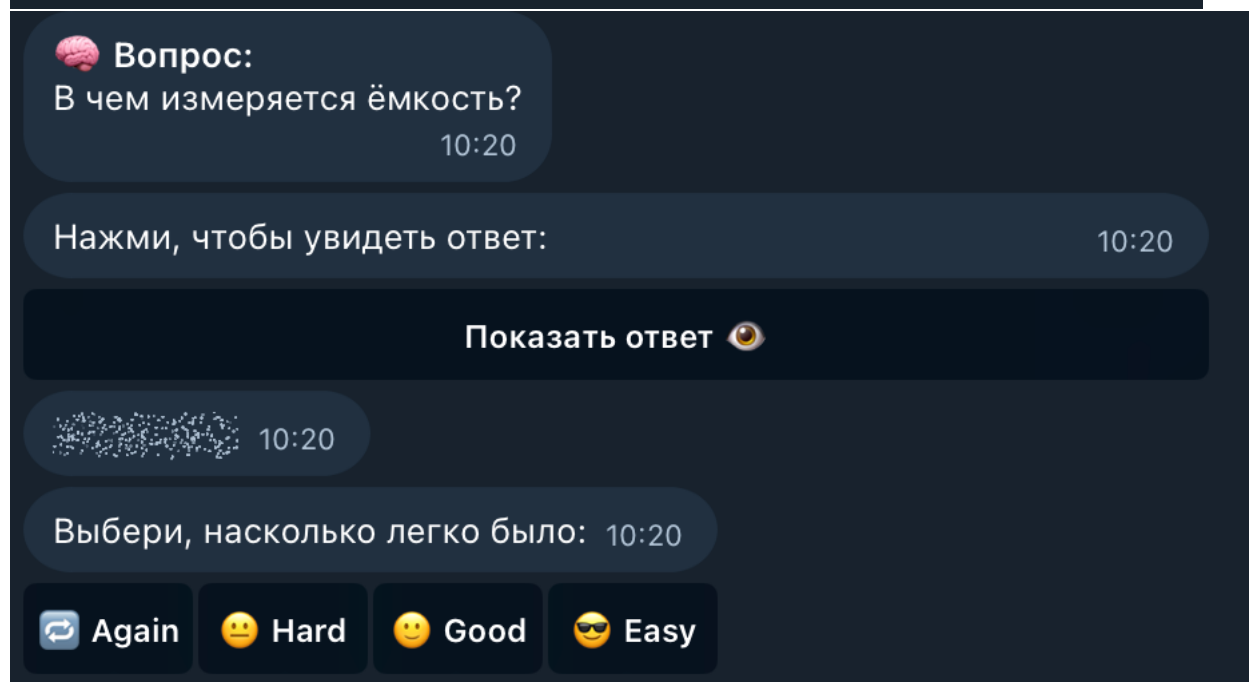
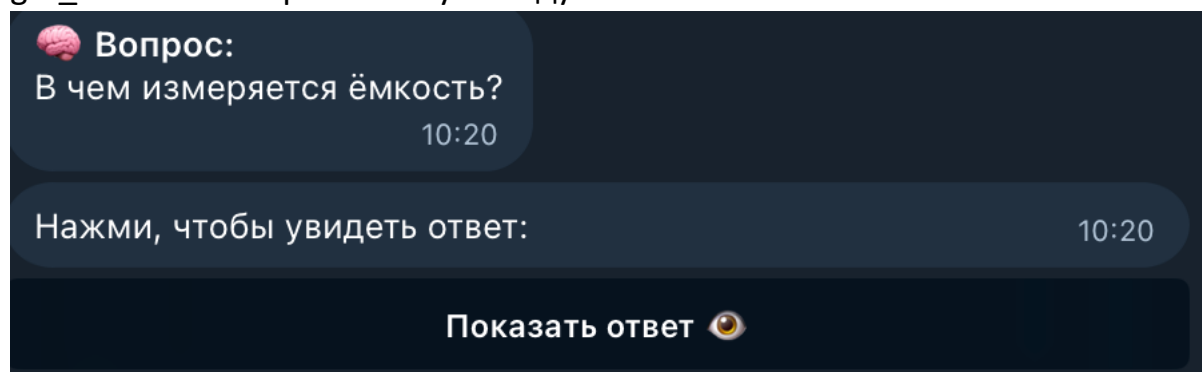
Запускаем anki.



(у нас присутствуют разные карточки для запоминания в колоде, которую мы создали в ANKI, и благодаря работе с anki.connect мы можем эти данные собирать)



Заходим в бота, если сессия не найдена, получаем колоды с помощью команды `get_decks` и выбираем нашу колоду `testbot`.



При нажатии кнопки ответ выдается спойлером, после этого система спрашивает насколько было легко (В дальнейшем планируется чуть более детальная работа с системой анализа ответов, более приближенная к алгоритму запоминания представленному в Anki)

